Laporan Kemajuan Mingguan
Pemetaan Fungsi Program dan Tuning Attention Patch Size pada Point Transformer V3

Nama: Nicholas Shane Pangihutan Siahaan
NIM : 23/520590/TK/57399
Program studi: Teknologi Informasi

16 September 2026

Proyek: Transformer-based Models for Geometric Deep Learning on Point-cloud Data
Periode eksperimen: 09–15 September 2026

1 Tujuan

Tujuan kegiatan minggu ini adalah:

1. Memetakan fungsi program, input, output, dan hubungan pemanggilan dalam pipeline PTv3 untuk klasifikasi ModelNet40.
2. Memahami serialization, proyeksi Query–Key–Value (QKV), patch attention, hierarchical pooling, dan update parameter.
3. Menguji pengaruh `enc_patch_size` dengan kondisi eksperimen lainnya tetap.
4. Mendokumentasikan hasil validation dan durasi training sebagai dasar eksperimen lanjutan.

Kegiatan ini merupakan *hyperparameter tuning*; model dilatih dari awal. Belum dilakukan fine-tuning dari pretrained checkpoint maupun modifikasi rumus QKV.

2 Pemetaan Struktur dan Fungsi Program

2.1 Struktur repository

`configs/` : Parameter model, dataset, optimizer, scheduler, dan hooks.
`pointcept/datasets/` : Membaca data, sampling, dan transformasi.
`pointcept/models/` : Backbone PTv3, classification head, dan loss.
`pointcept/engines/` : Training, validation, testing, dan hooks.
`pointcept/utils/` : Config parser, logging, registry, dan optimizer.
`libs/` : Operasi tambahan point cloud berbasis C++/CUDA.
`tools/` : Entry point Python dan program pembentukan split dataset.
`scripts/` : Peluncur shell untuk training dan testing.
`data/` : Dataset dan daftar split; pada workspace ini berupa symlink.
`exp/` : Log, checkpoint, TensorBoard, config, dan snapshot kode.

2.2 Hubungan pemanggilan

```
scripts/train.sh
  -> tools/train.py: main()
  -> default_config_parser() -> launch() -> main_worker()
  -> Trainer.__init__()
     -> model, dataset, DataLoader, optimizer, scheduler
  -> Trainer.train()
     -> dataset.__getitem__() -> transforms -> batch
     -> Trainer.run_step()
        -> DefaultClassifier.forward()
           -> PointTransformerV3.forward()
           -> mean aggregation -> classification head -> loss
        -> backward -> optimizer.step() -> scheduler.step()
     -> validation -> save checkpoint
  -> exp/modelnet40/<nama-eksperimen>/
```

`scripts/train.sh` menyalin source code ke `exp/.../code/` dan menjalankan snapshot tersebut. Perubahan source setelah training dimulai tidak otomatis mengubah proses yang sedang aktif.

2.3 Peta fungsi utama

Path yang diawali `datasets/`, `models/`, dan `engines/` pada tabel berikut berada di dalam folder `pointcept/`.

Tabel 1: Peta fungsi utama pipeline klasifikasi PTv3.

File / fungsi	Input dan proses	Output / efek
`tools/train.py:main()`	Membaca argumen CLI dan config; meluncurkan worker.	Proses training GPU.
`main_worker(cfg)`	Setup run; membangun trainer.	Memanggil `trainer.train()`.
`engines/defaults.py: default_config_parser()`	Menggabungkan config dan override; menetapkan dataset loop.	Config run dan salinannya.
`default_setup()`	Menetapkan seed serta batch/worker per GPU.	Environment eksperimen.
`datasets/modelnet.py: get_data_list()`	Membaca `modelnet40_<split>.txt`.	Daftar nama objek.
`ModelNetDataset.__init__()`	Root, split, jumlah titik, dan transform; membaca/membuat cache.	Dataset siap digunakan.
`get_data(idx)`	Cache atau file XYZ+normal; sampling titik; pembentukan label.	Dictionary koordinat, normal, kategori.
`__getitem__(idx)`	Memilih jalur persiapan train atau test.	Satu sampel.
`__len__()`	Jumlah objek dikalikan dataset loop.	Panjang dataset.
`prepare_train_data(idx)`	Mengambil data dan menerapkan transform.	Input train/validation.
`prepare_test_data(idx)`	Mengambil data dan membuat augmentasi test.	Voting list dan label.

File / fungsi	Input dan proses	Output / efek
`datasets/transform.py`	Normalisasi, scale, shift, grid sampling, konversi tensor.	Koordinat, grid coordinate, fitur dan label terpilih.
`models/default.py: DefaultClassifier.forward()`	Menjalankan backbone, mean aggregation per objek, head dan loss.	Loss saat train; logits dan loss sesuai mode evaluasi.
`models/utils/structure.py: Point.serialization()`	Grid coordinates, batch, dan metode ordering.	Serialized code, order, dan inverse.
`models/utils/serialization/default.py:encode()`	Memilih Z-order atau Hilbert; varian trans menukar x dan y.	Kode spasial integer.
`PointTransformerV3.forward()`	Serialization, sparsify, embedding dan encoder.	Fitur backbone dalam struktur Point.
`Embedding.forward()`	Fitur awal dan struktur sparse point cloud.	Fitur embedding awal.
`Block.forward()`	CPE, normalisasi, attention, MLP dan residual.	Fitur hasil transformer block.
`SerializedAttention.get_padding_and_inverse()`	Offset tiap objek dan patch size; membentuk padding.	pad, unpad dan cu_seqlens.
`SerializedAttention.forward()`	Fitur terurut; proyeksi QKV, attention patch dan proyeksi output.	Fitur hasil attention.
`MLP.forward()`	Linear, GELU, linear dan dropout.	Fitur hasil feed-forward network.
`SerializedPooling.forward()`	Pengelompokan hierarkis dan agregasi fitur.	Lebih sedikit titik dengan fitur stage berikutnya.
`SerializedUnpooling.forward()`	Upsampling dan skip features pada decoder.	Resolusi lebih tinggi; tidak digunakan saat `enc_mode=True`.
`engines/train.py: build_model()`	Membentuk model dari config dan menempatkannya di GPU.	Model trainable.
`build_train_loader()`, `build_val_loader()`	Membentuk dataset, sampler, dan DataLoader.	Iterator batch.
`build_optimizer()`, `build_scheduler()`	Parameter model dan config update bobot.	Optimizer dan scheduler.
`Trainer.train()`	Iterasi epoch/batch dan pemanggilan hooks.	Proses training lengkap.
`Trainer.run_step()`	Pemindahan batch ke CUDA, forward, backward dan update.	Bobot baru dan loss batch.
`hooks/evaluator.py: ClsEvaluator`	Prediksi pada validation loader.	allAcc, mAcc, F1 dan statistik kelas.
`hooks/misc.py:CheckpointSaver`	Epoch dan skor validation.	`model_best.pth`, `model_last.pth`.
`PreciseEvaluator.after_train()`	Checkpoint dan test config.	Testing otomatis setelah training.
`engines/test.py: ClsVotingTester.test()`	Model dan augmentasi objek test.	Hasil voting evaluation.

Komponen backbone pada tabel terdapat di `pointcept/models/point_transformer_v3/poin`

t_transformer_v3m1_base.py.

2.4 QKV dan serialized attention

Untuk fitur X, proyeksi attention secara konseptual ditulis sebagai

$$Q = XW_Q + b_Q, \quad K = XW_K + b_K, \quad V = XW_V + b_V. \tag{1}$$

Pada implementasi ini, ketiga proyeksi dikemas dalam satu layer linear `Linear(channels,3 *channels)`. Bobot proyeksi dipelajari selama training. Attention per head dalam satu patch secara konseptual adalah

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}}\right) V. \tag{2}$$

Fitur titik diurutkan menggunakan kode spasial sebelum dibagi menjadi patch. Flash Attention digunakan untuk menghitung attention pada pipeline aktif. Koordinat juga memengaruhi grid, ordering dan conditional positional encoding (CPE). Pada konfigurasi ini relative positional encoding eksplisit dinonaktifkan, yaitu `enable_rpe=False`.

3 Eksperimen

3.1 Dataset

Training set ModelNet40 terdiri atas 9.843 objek. Program `tools/create_modelnet_subset .py` memilih sekitar 10% objek dari setiap kelas secara acak dengan seed 42 dan pembulatan ke atas, menghasilkan 1.000 objek (sekitar 10,16% dari training set asli). Program `tools/sp lit_modelnet_train_val.py` membagi subset tersebut secara stratified sekitar 80:20 dengan pembulatan per kelas. Seluruh 40 kelas terwakili pada training dan validation. Kedua split tidak

Tabel 2: Pembagian dataset yang tetap untuk seluruh eksperimen.

Bagian	Objek	Nama split
Training	798	`train10_train`
Validation	202	`train10_val`
Official test	2.468	`test`

beririsan dan gabungannya sama dengan subset awal. Official test juga terpisah dari keduanya. Stratified random sampling mempertahankan distribusi kelas secara mendekati, tetapi tidak menjamin seluruh variasi geometri intra-kelas terwakili.

Pada level titik, farthest point sampling (FPS) mereduksi setiap objek menjadi 1.024 titik sebelum transformasi grid. Ini berbeda dari pemilihan subset antarobjek. Grid sampling selanjutnya dapat mengurangi jumlah titik aktual yang masuk ke model.

3.2 Environment dan konfigurasi tetap

Eksperimen dijalankan pada laptop dengan GPU NVIDIA GeForce RTX 3060 Laptop, Sistem Operasi Linux, Distro Ubuntu 22.04 dan VRAM 6 GB. Environment yang digunakan adalah `pointcept-torch2.5.0-cu12.4`: Python 3.10, PyTorch 2.5.0 dengan CUDA 12.4, spconv

2.3.8, Flash Attention 2.7.4.post1, dan torch-scatter 2.1.2. Versi ini berasal dari pemeriksaan environment proyek.

Parameter	Nilai
Backbone / task	PT-v3m1 / DefaultClassifier, 40 kelas
Mode	Encoder-only, `enc_mode=True`
Jumlah parameter	38.842.792 berdasarkan log
Input features	Koordinat XYZ dan normal XYZ, total 6 channel
Seed model / split	42 / 42
Epoch / eval_epoch	100 / 100; train.loop = 1
Batch train / validation	4 / 8
Worker	2
Optimizer	AdamW; LR dasar 0,001; weight decay 0,01
LR kelompok block	0,0001; sesuai `param_dicts`
Scheduler	OneCycleLR; max LR kelompok 0,001 dan 0,0001
Loss	Cross Entropy + Lovasz multiclass, masing-masing bobot 1
Ordering	z, z-trans, hilbert, hilbert-trans; shuffle aktif
Regularisasi	drop_path = 0,3; attention dropout = 0
Flash / AMP	Flash Attention aktif; AMP global nonaktif

Meskipun AMP global nonaktif, implementasi Flash Attention mengonversi QKV ke dtype yang didukung kernel di dalam modul attention.

3.3 Variabel tuning dan kontrol

Variabel yang diubah adalah `enc_patch_size`, diterapkan sama pada kelima stage encoder. Nilai kandidat adalah 128, 256, dan 512; baseline menggunakan 1024. Semua model dimulai dari random initialization tanpa memuat checkpoint baseline. Dataset, optimizer, scheduler, seed, jumlah epoch, ordering, dan parameter model lainnya tetap.

Untuk mencegah testing otomatis selama tuning, config induk `cls-ptv3-v1m1-0-laptop-tuning.py` menghilangkan hook `PreciseEvaluator`. Validation dan checkpoint tetap berjalan. Baseline sebelumnya memang menjalankan official test setelah training, tetapi angka official test tidak digunakan untuk memilih patch size.

3.4 Metrik dan durasi

Metrik pemilihan model adalah overall accuracy pada validation:

$$\text{allAcc} = \frac{\sum_{c=1}^{40} n_{c,\text{correct}}}{\sum_{c=1}^{40} n_c}, \qquad \text{mAcc} = \frac{1}{40}\sum_{c=1}^{40} \frac{n_{c,\text{correct}}}{n_c}. \tag{3}$$

Macro F1 merupakan rata-rata F1 per kelas. Setiap model dipilih pada epoch dengan validation allAcc tertinggi; jika skor sama, tabel menggunakan kemunculan pertama. mAcc dan macro F1 dilaporkan pada epoch terpilih, bukan maksimum terpisah sepanjang training.

Durasi dihitung dari timestamp `StartTraining` sampai hasil validation epoch ke-100. Durasi mencakup training dan validation berkala, tetapi tidak mencakup persiapan dataset, voting test, atau penulisan checkpoint terakhir setelah validation. Durasi bukan benchmark inference.

4 Hasil Eksperimen

Semua run selesai 100 epoch tanpa error yang tercatat di log. Setiap run menghasilkan `model_best.pth` dan `model_last.pth`. Pada tabel, m dan d berarti menit dan detik; waktu dibulatkan

Tabel 3: Hasil pada validation set. Seluruh skor dalam persen.

Patch	allAcc	mAcc	Macro F1	Epoch	Durasi
1024	**75,25**	60,81	57,50	73	47 m 36 d
128	73,76	57,39	53,08	83	47 m 15 d
256	74,75	**61,14**	57,45	80	47 m 55 d
512	74,26	60,75	**58,90**	95	48 m 25 d

ke detik terdekat. Skor dibulatkan dari log. Dengan 202 objek validation, jumlah prediksi benar pada epoch terbaik adalah 152, 149, 151, dan 150 untuk patch 1024, 128, 256, dan 512 secara berurutan.

5 Analisis dan Keterbatasan

5.1 Akurasi

Baseline patch 1024 memberikan allAcc tertinggi. Patch 128, 256, dan 512 lebih rendah sekitar 1,49; 0,50; dan 0,99 poin persentase. Patch 256 menjadi kandidat tuning paling dekat dengan baseline, dan mAcc-nya sedikit lebih tinggi. Patch 512 memiliki macro F1 tertinggi, menunjukkan bahwa peringkat konfigurasi dapat berbeda menurut metrik. Pemilihan utama tetap mengikuti allAcc yang telah ditetapkan.

Satu prediksi benar tambahan pada validation set mengubah allAcc sebesar $100/202 \approx 0,495$ poin persentase. Selisih baseline dan patch 256 hanya satu objek. Oleh karena itu, hasil ini tidak cukup untuk membuktikan superioritas statistik konfigurasi tertentu.

5.2 Efisiensi

Durasi seluruh run berada pada rentang 47 menit 15 detik sampai 48 menit 25 detik. Patch 128 sekitar 21 detik lebih cepat dari baseline, sedangkan patch 256 dan 512 sedikit lebih lambat. Pengecilan patch belum memberikan penghematan waktu training yang berarti pada pengukuran ini.

Hipotesis penjelasannya adalah bahwa input hanya 1.024 titik dan pooling mengurangi jumlah titik pada stage berikutnya; sebagian stage mungkin memiliki jumlah titik di bawah batas patch. Selain itu, durasi total dipengaruhi sparse convolution, DataLoader, validation, dan penyimpanan checkpoint, bukan attention saja. Hipotesis ini memerlukan profiling untuk dibuktikan. Kompleksitas attention konvensional per patch $O(P^2)$ tidak langsung menentukan keseluruhan waktu pipeline Flash Attention pada eksperimen ini.

5.3 Batas validitas

1. Seluruh konfigurasi baru diuji dengan satu seed training.
2. Validation set kecil, khususnya kelas minoritas yang hanya memiliki satu sampai beberapa objek; mAcc dapat berfluktuasi.

3. Subset mempertahankan distribusi kelas, bukan jaminan keragaman geometri lengkap dari dataset asli.
4. Waktu diukur dari log pada sesi berbeda, tanpa kontrol ketat suhu, daya GPU, dan proses latar belakang.
5. Peak VRAM, latency inference, dan profiler attention belum diukur.
6. Kurva learning dan confusion matrix belum dianalisis dalam laporan ini. Tidak disajikan grafik sintetis atau data yang belum diekstrak.

Angka hasil tidak dapat langsung dibandingkan dengan benchmark paper PTv3 karena dataset, subset, tugas, dan protokol eksperimennya berbeda.

6 Kesimpulan dan Rencana Selanjutnya

Pipeline program telah dipetakan dari peluncur training sampai dataset, backbone, classification head, loss, update bobot, validation, dan checkpoint. Eksperimen ini berhasil menguji satu hyperparameter dengan kondisi lain tetap dan tanpa official test otomatis pada run tuning.

Pengecilan patch belum meningkatkan validation allAcc maupun memberi percepatan total yang berarti. Baseline 1024 tetap menjadi kontrol utama. Patch 256 disimpan sebagai kandidat konfirmasi karena hasilnya dekat dengan baseline dan mAcc sedikit lebih tinggi. Temuan ini merupakan hasil sementara, bukan klaim bahwa patch 1024 selalu optimal.

Rencana selanjutnya:

1. Menguji ordering Z, Hilbert, dan kombinasi Z+Hilbert dengan patch 1024 serta parameter lain tetap.
2. Mengulangi baseline dan konfigurasi menjanjikan dengan seed 123 dan 501; melaporkan mean dan standard deviation.
3. Mengekstrak kurva loss/validation dan memprofilkan komponen waktu serta penggunaan VRAM.
4. Melakukan tuning learning rate/regularisasi setelah ablation utama.
5. Melakukan fine-tuning hanya setelah sumber pretrained checkpoint dan protokol pembanding ditetapkan.
6. Menjalankan official test pada model terpilih dengan aturan voting yang sama dan telah ditetapkan; tidak memilih hasil voting berdasarkan skor test tertinggi saja.

Daftar Pustaka

[1] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, dan H. Zhao. *Point Transformer V3: Simpler, Faster, Stronger*. CVPR, 2024. `https://arxiv.org/abs/2312.10035`.

[2] Pointcept contributors. *Pointcept: A Codebase for Point Cloud Perception*. Repository source code. `https://github.com/Pointcept/Pointcept`.

[3] Artefak eksperimen lokal. Config, snapshot source, checkpoint, dan log baseline serta tuning patch size, 13–15 September 2026. Path artefak dicantumkan pada Lampiran A.